BEST Agentic AI Summer 2026
PBL Project Guideline

Preliminaries

According to Startup Genome, 90% of all technology startups fail. Among the most common reasons are a lack of market need due to insufficient research, poor competitor analysis, and fundamentally flawed business models. While no methodology can guarantee success, carrying out a proper Conceptual Design of a Software Application (PCAS) helps determine whether a project is worth pursuing before significant development effort is invested.

This workshop introduces a structured way of approaching software projects through **Problem-Based Learning (PBL)**. PBL is a learning model designed to help participants investigate and solve authentic problems, ranging from challenges faced by individuals to issues affecting entire communities. Rather than focusing solely on producing a final solution, PBL emphasizes the reasoning and decision-making process used to reach it.

Several principles define the PBL approach. Throughout a project, teams are encouraged to continuously question both the problem they are solving and the solution they are developing. Regular critique and revision help identify incorrect assumptions early, while reflection supports learning by encouraging participants to evaluate both their progress and their decision-making process. Many of these principles closely resemble the activities involved in the early stages of developing a successful startup.

The goal of this workshop is therefore twofold. First, it provides an opportunity to experience working within a PBL environment. Second, it helps participants develop a structured approach to planning and designing software projects, increasing the likelihood that their ideas address genuine problems and can be developed into meaningful solutions.

Unlike traditional coursework, where both the problem and the expected solution are often predefined, PBL places much greater responsibility on the learners. During this workshop, your team will identify its own problem within the overall theme of **agentic AI**. Your work will be evaluated not only on the final outcome, but also on how you explored ideas, justified your decisions, dealt with uncertainty, and adapted your approach throughout the project.

This workshop follows the **Aalborg model** of Problem-Based Learning. Three characteristics define this approach:

- Your team chooses its own problem within the workshop theme of **agentic AI**.
- You work collaboratively in a small group rather than individually.
- The entire workshop revolves around a single project, from the initial idea to the final demonstration.

Agentic AI is particularly well suited to this style of learning. As a rapidly evolving field, there is rarely a single correct solution or development process. Success depends on exploring

alternatives, experimenting with different approaches, and making informed decisions rather than following a fixed recipe. PBL therefore mirrors the way many real-world AI projects are developed.

Especially during the first day, it is perfectly normal not to know exactly what you want to build. Uncertainty is an expected part of the process rather than a sign that something is wrong. Your supervisors will not provide ready-made answers or tell you which direction to take. Instead, they will ask questions that encourage your team to think critically, justify its decisions, and take ownership of both the problem and the proposed solution.

This guideline provides the structure necessary to keep the project organised without limiting your creativity. Certain aspects of the workshop are fixed, including the daily schedule, the required deliverables, and the evaluation criteria. Within that framework, however, your team is responsible for choosing the problem, defining the solution, selecting the technologies, and organising its own workflow. Simply following the chapter structure is not sufficient for successful PBL—the quality of your reasoning and the decisions you make throughout the project are equally important.

Although formal project roles are not required, teams are encouraged to establish a few simple working agreements early in the workshop. For example, discuss how and when the team will communicate during the day, how disagreements will be resolved, and who will ensure that the group maintains a reasonable pace and does not spend too much time on a single task.

Finally, remember that this is a collaborative project. Every team member should understand the complete solution, including parts they did not personally implement. Not everyone will have previous programming experience, and that is entirely expected. There are valuable contributions beyond writing code, including research, testing, documentation, presentation preparation, and project organisation. Make sure everyone has the opportunity to contribute, and always communicate respectfully with both your teammates and your supervisor.

Getting Started

The workshop begins by forming project teams and preparing the basic resources you will use throughout the following four days. Spending a little time organizing yourselves at the beginning will make collaboration much smoother later on.

Pitching. The project starts with a pitching and team formation session. During this activity, you will present and discuss initial ideas with other participants, helping you identify people who are interested in solving similar problems. The goal is not to choose a final solution immediately, but to form a team around a shared area of interest that you can explore together.

Set up shared materials. Once your team has been formed, prepare a few shared resources that will be used throughout the project:

- A **problem log** — a simple document where you record every potential problem your team identifies during the exploration phase, including ideas that you later decide not to

pursue.

- A **project journal** — a running record of your team's decisions, research findings, and progress across all four workshop days.
- A **shared workspace** — a common location where all project files, notes, screenshots, and other materials can be stored and accessed by every team member, regardless of their role.

These resources will not only help your team stay organized but will also provide useful material for your final presentation and retrospective.

TODO: add links/templates for the problem log, project journal, and shared workspace once decided.

Work with your supervisor. Each team is assigned a supervisor who is available throughout every workshop day. You do not need to schedule meetings in advance. Whenever your team is uncertain, needs feedback, or wants to discuss an idea, take the initiative and ask for guidance. Reaching out to your supervisor is your responsibility, not theirs.

What to expect from your supervisor. The supervisor's role is to support your learning rather than direct your project. Instead of providing answers, they will encourage your team to think critically and reflect on its decisions. In particular, your supervisor may:

- Challenge your assumptions by asking questions such as: "How do you know this is a real problem?" or "Why did you choose this approach over the alternatives?"
- Ask you to explain and justify your design decisions.
- Recommend useful resources, references, or people who may help your investigation.
- Help your team recognize when you are overthinking a problem and need to make a decision to move forward.

At the same time, there are things your supervisor deliberately will *not* do. They will not choose your problem for you, approve or reject your problem statement, solve technical implementation issues, or fix your code. If you ask questions of this nature, expect them to respond with further questions instead of direct answers. This is an intentional part of the PBL process, where learning comes from making and reflecting on your own decisions.

Agree on communication. Before beginning your project, agree on how your team will communicate both internally and with your supervisor. For example, decide whether you will use a shared chat, hold short daily check-ins, or adopt another communication method that suits your team. Establishing clear communication practices from the outset helps ensure that everyone remains informed, involved, and able to contribute throughout the workshop.

The Task: Explore the Theme and Find Your Problem

The first day is dedicated to understanding the problem you want to solve. Rather than immediately designing a solution, your team should spend time exploring the theme of agentic AI, identifying real-world problems, and validating that the challenge you choose is both meaningful and achievable within the workshop.

By the end of the day, your team should have:

- Explored the theme of agentic AI and identified potential application areas.
- Chosen and documented a clear problem to solve.
- Experimented with different AI tools and technologies without committing to a specific solution.
- Recorded an individual reflection on the day's work.

Explore the theme. Begin by discussing the kinds of problems that agentic AI could help solve. Resist the temptation to jump directly to a solution or a particular technology. Instead, focus on understanding the problem space and identifying situations where autonomous AI systems could provide genuine value.

As your discussion progresses, use the **problem log** introduced in Chapter 2 to record every problem your team considers, including ideas that you eventually decide not to pursue. Keeping track of discarded ideas helps document your reasoning and demonstrates how your final choice emerged.

Find your problem. After exploring several possibilities, agree on a single problem that your team will address during the remainder of the workshop. Summarize it in a short and clear **Problem Statement**.

A good problem statement should:

- clearly identify who experiences the problem;
- explain why it is a genuine problem rather than simply describing a desired feature;
- be realistic to investigate and address within the remaining three workshop days; and
- describe the problem itself without suggesting or prescribing a particular solution.

Remember that your problem statement may evolve as you learn more about the domain. Refining your understanding is a normal and expected part of the PBL process.

Try simple tools. Spend some time experimenting with different AI tools or models to better understand their capabilities and limitations. At this stage, these experiments are exploratory rather than definitive—nothing you try today has to become part of your final project.

This activity is valuable for every team member, regardless of technical background. Examples include experimenting with conversational AI systems, testing no-code AI platforms, or observing how changing prompts affects an AI's behaviour.

Prepare for later. Before finishing the day, write down—in simple, clear language—*why* the problem you selected is worth solving. This explanation will become an important part of your final presentation, helping you communicate the motivation behind your project.

Individual reflection. Conclude the day by taking a few minutes to reflect individually. Consider what you learned, what you found challenging, and how you currently feel about the direction of your project. These reflections are intended to help you evaluate your own learning throughout the workshop.

TODO: add a link where students can write their daily reflection.

The Task: Research and Design

With a well-defined problem in place, the second day focuses on planning your solution. Before you begin building, take time to understand the problem in greater depth, investigate possible approaches, and make deliberate design decisions. A well-researched and carefully scoped design will make implementation on the following day much more straightforward.

By the end of the day, your team should have:

- Researched both the problem and potential solution approaches.
- Decided on an appropriate level of complexity for the project.
- Documented the main design decisions behind your solution.
- Prepared the tools and workspace needed for implementation.
- Recorded an individual reflection on the day's work.

Research the problem. Build upon the work completed on Day 1 by learning more about the people affected by your chosen problem and why it is important to them. Understanding the users and their needs will help ensure that your solution addresses a genuine challenge rather than an assumed one.

This stage does not require programming knowledge, making it an opportunity for every team member to contribute. Reading articles, exploring existing solutions, interviewing potential users, or discussing the problem with others are all valuable forms of research.

Research the solution. Once you have a solid understanding of the problem, investigate possible ways of solving it. Explore existing tools, frameworks, and AI technologies that could support your solution, and compare their strengths and limitations.

As you evaluate alternatives, consider not only what is technically possible, but also what your team can realistically learn, implement, and demonstrate within the remaining time available.

Choose your complexity level. Using the *Complexity Ladder* introduced earlier in the workshop, decide how ambitious your project should be. A successful project is not necessarily the most complex one—it is the one that solves its chosen problem effectively while remaining achievable.

When making this decision, discuss questions such as:

- What is the intended goal of the system, and how will it determine whether it has achieved that goal?
- Which tools, data sources, or external services will it need?
- How should the system respond if it encounters an error or cannot complete its task?
- Is this level of complexity realistic given the remaining time and your team's experience?

Whenever possible, choose the simplest solution that adequately addresses your problem. Simplicity is often a strength rather than a limitation.

Make your design decisions. Agree as a team on the overall design of your solution and document the reasoning behind your choices. Decide which technologies you will use, how the different parts of the system will interact, and what responsibilities each component will have.

A useful way to evaluate your scope is to ask yourselves:

If one part of our design were removed, would the system still demonstrate agentic behaviour—that is, would it still be able to plan, act, evaluate its progress, and adapt?

If the answer is yes, consider simplifying your design by removing that component. If removing it fundamentally changes the nature of the system, it is likely an essential part of your solution.

Set up your workspace. Before ending the day, prepare everything needed for implementation. This may include creating project repositories, configuring development environments, setting up user accounts, or organizing your shared workspace so that everyone can contribute efficiently once development begins.

Prepare for later. Record your most important design decisions together with the reasoning behind them. These notes will become valuable material for your final presentation, where you will need to explain not only what you built, but also why you built it that way.

Individual reflection. Conclude the day with an individual reflection. Consider what you learned, what you found challenging, and how confident you feel about the design your team has produced.

The Task: Build

With the research and design completed, the third day is dedicated to turning your plans into a working solution. Focus on implementing the core functionality first, working effectively as a team, and documenting your progress as you go. The objective is not to build every feature you can imagine, but to develop a reliable demonstration that addresses the problem you identified.

By the end of the day, your team should have:

- Built the core functionality of the proposed solution.
- Distributed responsibilities effectively among team members.
- Documented the current state of the project with supporting evidence.
- Recorded an individual reflection on the day's work.

Build. Begin implementing the solution you designed on Day 2. Prioritize the essential components that demonstrate the value of your idea before investing time in additional features or refinements. A simple solution that works reliably is far more valuable than an ambitious solution that remains unfinished.

As development progresses, continue referring back to your problem statement and design decisions. If new ideas arise, note them down, but avoid expanding the project's scope unless the changes are necessary to solve the original problem.

Divide the work. Organize the workload so that every team member has a clear responsibility while maintaining a shared understanding of the project as a whole. Collaboration should extend beyond implementation—everyone should remain familiar with the overall design and progress of the system.

Not every contribution needs to involve programming. Team members who are not writing code can make valuable contributions by testing the system, documenting bugs and improvement ideas, maintaining the project journal, preparing presentation material, researching implementation questions, or helping coordinate the team's work.

Save your progress. As major components begin to work, capture evidence of your progress by taking screenshots or recording short video clips. Store these materials in your shared workspace together with any relevant documentation.

Collecting this material throughout the day is much easier than trying to recreate it later, and it will provide useful evidence for your final presentation and demonstration.

Individual reflection. At the end of the day, take a few minutes to reflect individually on your work. Consider what you learned, what challenges you encountered, how your team overcame them, and how confident you feel about the current state of your project.

The Task: Final Touches, Presentation, and Retrospective

The final day is dedicated to bringing your project to a close. Rather than introducing new functionality, your team should focus on ensuring that the existing solution works as intended, preparing a clear presentation, and reflecting on the overall learning experience. By this point, the emphasis shifts from development to demonstrating and communicating the work you have completed over the previous three days.

By the end of the day, your team should have:

- Completed any essential fixes needed to stabilize the project.
- Presented and defended the project before an audience.
- Conducted a group retrospective on the workshop experience.
- Recorded a final individual reflection.

Final touches. Use the beginning of the day to verify that your project behaves as intended and addresses the problem described in your original Problem Statement. Focus only on fixing existing issues, improving stability, and ensuring that your demonstration is reliable.

Avoid adding new features at this stage. New functionality often introduces new bugs and leaves insufficient time for testing, making the final presentation more difficult rather than more impressive.

Present and defend. Your team will give a live demonstration of the completed project, followed by a question-and-answer session. Be prepared to explain not only *what* your system does, but also *why* you made the decisions that shaped it.

Every team member should be able to answer questions about the project as a whole, not only the parts they personally implemented. Developing a shared understanding throughout the workshop is therefore essential.

Your presentation should build directly upon the work your team has already documented. In particular, the explanation of *why the problem matters* from Day 1 and the record of your key *design decisions* from Day 2 provide the foundation for your presentation narrative. If these materials were prepared carefully during the workshop, much of your presentation is already written.

Retrospective. After the presentations, take time as a team to reflect on the workshop as a whole. Discuss what went well, what challenges you encountered, how your team responded to them, and what you would approach differently in a future project. The objective is not

to judge success or failure, but to identify lessons that will help you in future collaborative projects.

Individual reflection. Conclude the workshop with a final individual reflection. Think about what you learned during the four days, what surprised you, which skills you developed, and how your understanding of both Problem-Based Learning and agentic AI has evolved throughout the project.

Reporting

Evaluation

Your project will be evaluated during the final presentation. The assessment considers both the quality of the solution you developed and the process your team followed throughout the workshop. In line with the principles of Problem-Based Learning, a successful project is not measured solely by the sophistication of the final prototype, but also by the reasoning behind your decisions, your collaboration as a team, and your ability to reflect on the challenges you encountered.

The evaluation criteria are summarized below.

Criterion	What this means
Problem	Did your team clearly identify and understand a meaningful problem? Did you investigate it thoroughly, identify the target users, and demonstrate why it is worth solving?
Solution	Does your proposed solution address the identified problem effectively? Are your design decisions appropriate, and did you select tools and technologies that are justified by your project's goals and constraints?
Demo	Did your demonstration work as intended? Did it clearly show the functionality your team claimed to have implemented?
Presentation	Was your presentation clear, well structured, engaging, and easy to follow? Did it explain both the motivation behind the project and the reasoning behind your design decisions?
Working process	Did your team follow the Problem-Based Learning process described in this guideline? Did you make effective use of your supervisor, complete the daily reflections, and collaborate constructively when making decisions or overcoming obstacles? A team that encounters genuine difficulties but responds thoughtfully, documents its reasoning, and adapts its approach will be evaluated positively on this criterion, even if the final prototype remains limited as a result.

Remember that the objective of this workshop is not simply to produce a functioning agentic AI application, but to develop a structured approach to solving complex problems collaboratively. Demonstrating thoughtful decision-making, effective teamwork, and continuous reflection is just as important as the final technical outcome.

Good Luck!